

Links, buttons, and states

In the [HTML pack](#), you learned the difference between links and buttons: a link navigates somewhere, while a button performs an action.

CSS can make both of them look polished, but it should not erase that difference. A link styled like a button is still a link. A button styled like plain text is still a button.

This lesson is about styling interactive elements and their states: normal, hover, focus, active, visited, and disabled.

Start with an interactive page

Use a small interactive page with navigation links, a text link, a link styled like a primary action, and a few buttons:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Links and buttons practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Links, buttons, and states</h1>
        <p class="summary">Interactive elements should look clickable, show focus, and r
      </section>

      <nav class="site-nav" aria-label="Practice navigation">
        <a href="/lessons">Lessons</a>
        <a href="/projects">Projects</a>
        <a href="/account">Account</a>
      </nav>

      <article class="panel">
        <p class="badge">Draft lesson</p>
        <h2>Publish your CSS notes</h2>
        <p class="panel-text">
          Review the <a href="/style-guide">style guide</a> before publishing.
          If the lesson is ready, use the action buttons below.
        </p>

        <div class="actions">
          <a class="primary-link" href="/preview">Preview lesson</a>
          <button class="secondary-button" type="button">Save draft</button>
          <button class="danger-button" type="button">Delete draft</button>
          <button class="secondary-button" type="button" disabled>Syncing</button>
        </div>
      </article>
    </main>
```



```
</body>
</html>
```

Then add the starting styles:

CSS

```
* {
  box-sizing: border-box;
}

body {
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f9fafb;
}

.page {
  max-width: 760px;
  margin: 40px auto;
}

.intro,
.panel {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  padding: 24px;
}

.intro,
.site-nav {
  margin-bottom: 24px;
}

.eyebrow,
.site-nav a {
  color: #2563eb;
  font-weight: 700;
}

.summary,
.panel-text {
  color: #4b5563;
  line-height: 1.6;
}

.badge {
  display: inline-block;
  color: #1d4ed8;
```

```
border-radius: 999px;
font-size: 14px;
font-weight: 700;
}

a {
  color: #2563eb;
  text-decoration-thickness: 2px;
  text-decoration-color: #2563eb;
  text-decoration: underline;
}

.site-nav a {
  display: inline-block;
  margin-right: 12px;
  text-decoration: none;
}

.actions {
  margin-top: 20px;
}

.primary-link,
```

```
.secondary-button,
.danger-button {
  display: inline-block;
  margin-right: 8px;
  margin-bottom: 8px;
  padding: 10px 14px;
  border-radius: 6px;
  font-family: inherit;
  font-size: 16px;
  font-weight: 700;
  text-decoration: none;
}

.primary-link {
  color: white;
  background-color: #2563eb;
}

.secondary-button,
.danger-button {
  background-color: white;
  cursor: pointer;
}

.secondary-button {
  color: #2563eb;
  border: 1px solid #bfdbfe;
}

.danger-button {
  color: #b91c1c;
  border: 1px solid #fecaca;
}
```

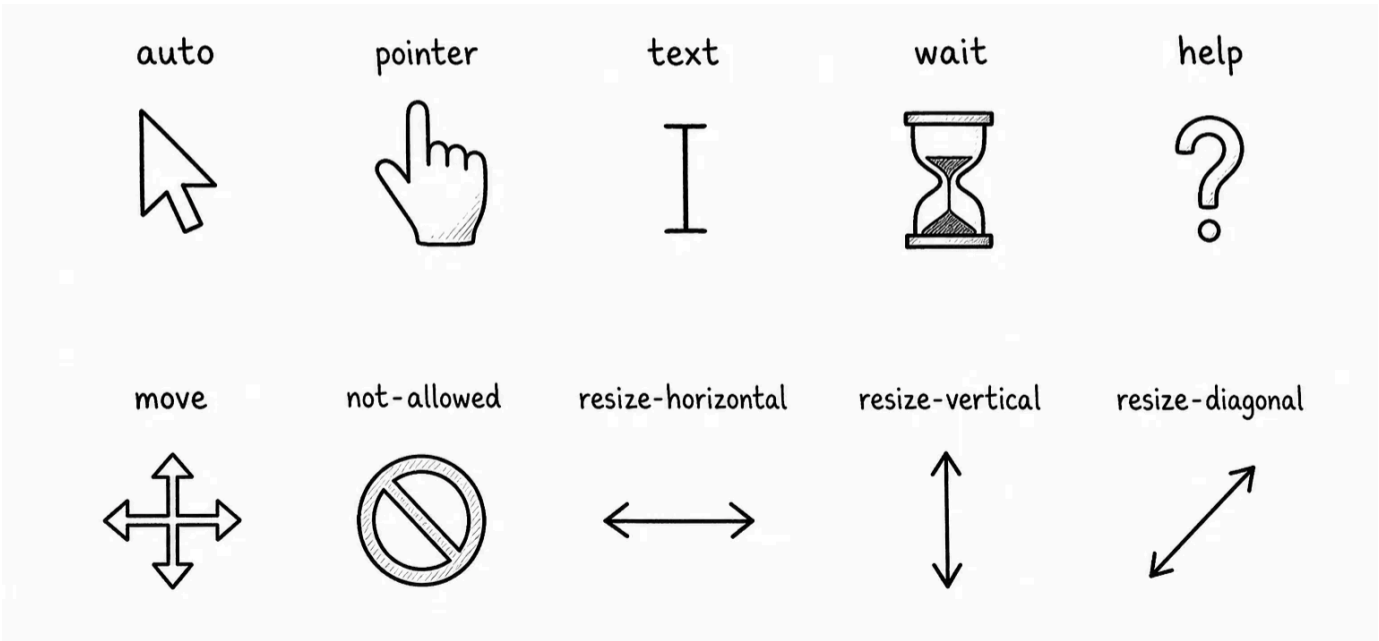
Open the page and you should see normal links, navigation links, a link styled like a primary action, regular buttons, a danger button, and a disabled button.

Right now the controls have base styles only. The page can show what each control is, but it does not yet show how each control responds when someone hovers, tabs, clicks, or reaches a disabled state.

Cursors

If you look closely at the CSS file, you will notice the `cursor` property. We have not used it much yet in this pack. The `cursor: pointer` declaration gives the element the familiar hand pointer.

The image below shows some possible `cursor` values and how each one looks. You can [look at this page](#) for more details about the property and its values.



Links and buttons have different jobs

Start by choosing the right element for the job.

In the practice HTML, links are used for navigation. For example:

html

```
<a href="/preview">Preview lesson</a>
```

Buttons are used for actions, such as saving the current draft:

html

```
<button type="button">Save draft</button>
```

CSS can make the link look like a button:

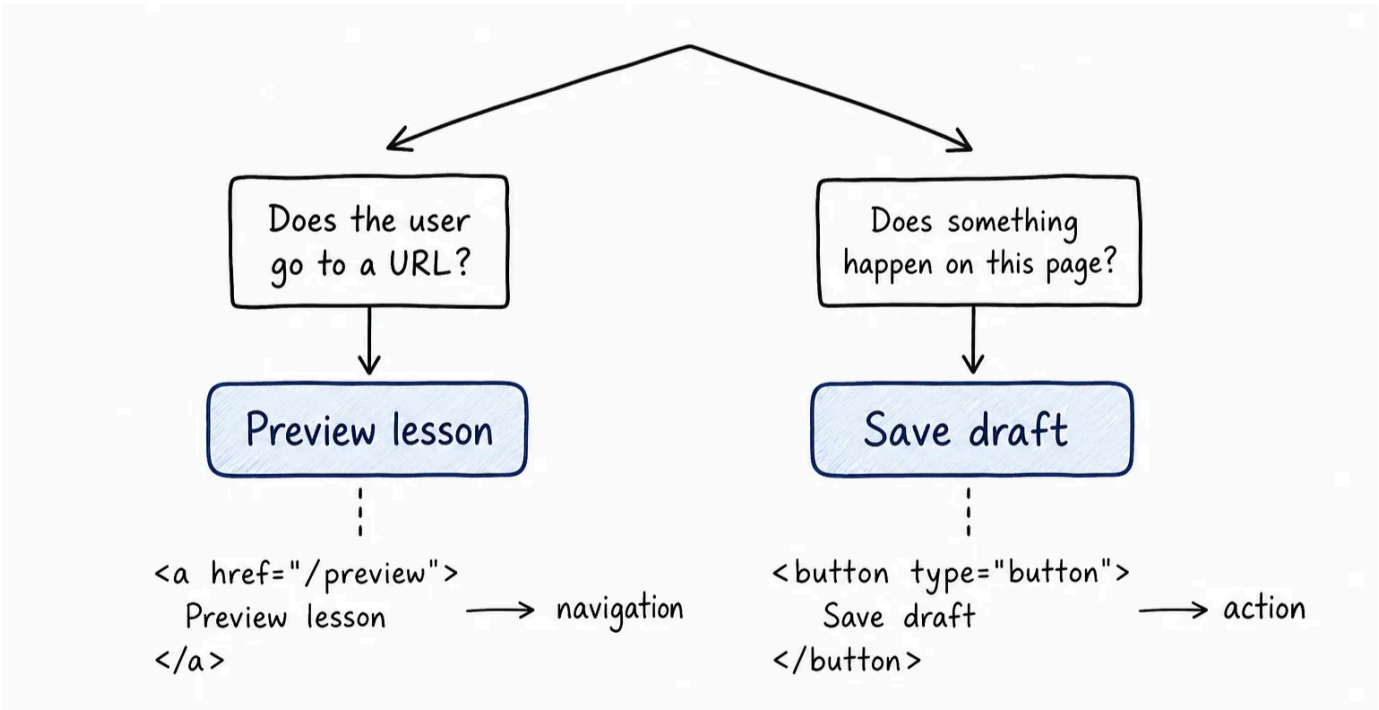
CSS

```
.primary-link {
  color: white;
  background-color: #2563eb;
}
```

But it is still a link because the HTML is `<a href="/preview">` , so it navigates to a URL.

In our example file, the save and delete controls are buttons because they represent actions on the current page.

That choice matters for keyboard behavior, screen readers, browser features, and forms.



💡 **Choose meaning first** - Decide whether the user is navigating or performing an action before styling the element. Use `<a>` for navigation and `<button>` for actions, then use CSS to make it fit the design.

Keep links recognizable

Links should look like links, especially inside paragraphs.

The base rule keeps paragraph links underlined and adjusts the underline:

CSS

```
a {
  color: #2563eb;
  text-decoration-thickness: 2px;
  text-decoration-offset: 3px;
}
```

`text-decoration-thickness` controls how thick the underline is. `text-decoration-offset` controls how far the underline sits from the text. Together, they keep the link recognizable while making the underline feel more deliberate than the browser default.

For navigation links, removing the underline can be okay because their placement and grouping already signal that they are links:

CSS

```
.site-nav a {
  display: inline-block;
  margin-right: 12px;
  text-decoration: none;
}
```

If you remove an underline, make sure the link still looks clickable through color, placement, spacing, or another clear cue.

⚠ **Do not hide links in plain text** - If a link inside a paragraph has the same color and decoration as the surrounding text, people may not notice it, so keep body links visibly link-like.

State selectors match interaction states

A state selector is a selector that matches an element in a specific condition.

The practice page currently has base styles, but it does not have state styling yet. The controls do not visibly respond when someone hovers, tabs, presses, or reaches a disabled state. CSS handles these conditions with pseudo-classes.

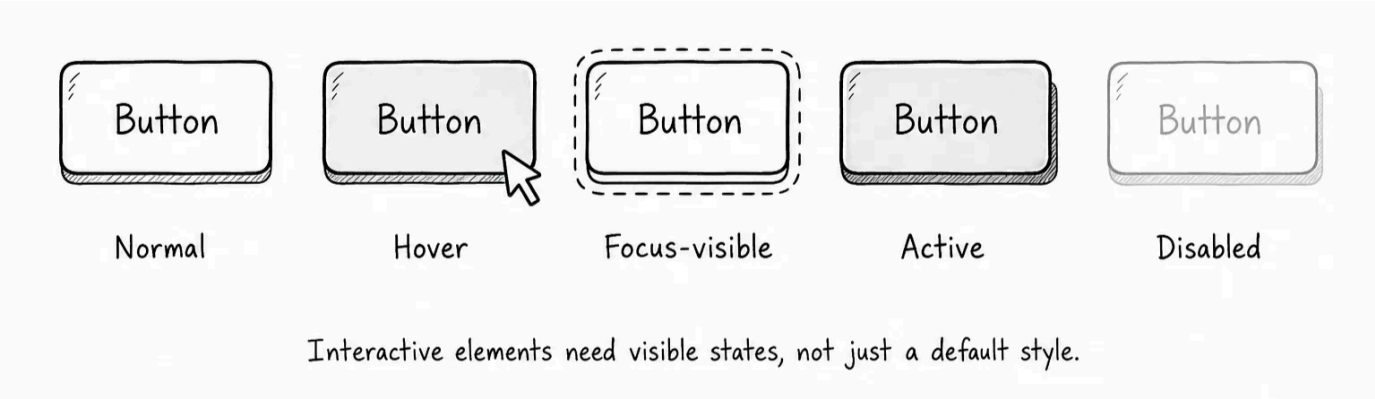
For example, `:hover` is a pseudo-class that matches when the pointer is over an element:

CSS

```
a:hover {
  color: #1d4ed8;
}
```

The colon is the clue: selectors like `:hover`, `:focus-visible`, `:active`, `:visited`, and `:disabled` are called pseudo-classes.

The image below shows common states an interactive control can have. The next sections style those states one at a time:



State selectors are still selectors, so they follow the cascade from Lesson 7. Source order still matters when two state rules set the same property on the same element.

Add hover states

`:hover` matches when the user's pointer is over an element. Add these hover styles:

CSS

```
a:hover {
  color: #1d4ed8;
}
```



```
.primary-link:hover {
  color: white;
  background-color: #1d4ed8;
}

.secondary-button:hover {
  background-color: #eff6ff;
}

.danger-button:hover {
  background-color: #fef2f2;
}
```

Reload the page and move your pointer over the links and buttons.

The first rule changes the color of normal links on hover. The other rules change the background color of the button-like controls. The link styled as `.primary-link` also keeps `color: white;` in its hover rule so the text stays readable on the darker blue background.

Hover states should be noticeable, but they do not need to be dramatic. A small color or background change is usually enough.

Hover is useful for pointer users. It is not enough on its own because keyboard and touch users may never see it.

Add keyboard focus states

Focus is the state an element enters when it is selected for keyboard interaction.

Press `Tab` in the browser, and the focus should move through the navigation links, the action link, and the buttons.

The disabled `Syncing` button should be skipped. Disabled controls cannot receive focus, which is one reason they need a clear disabled style.

Use `:focus-visible` for a keyboard-friendly focus style:

CSS

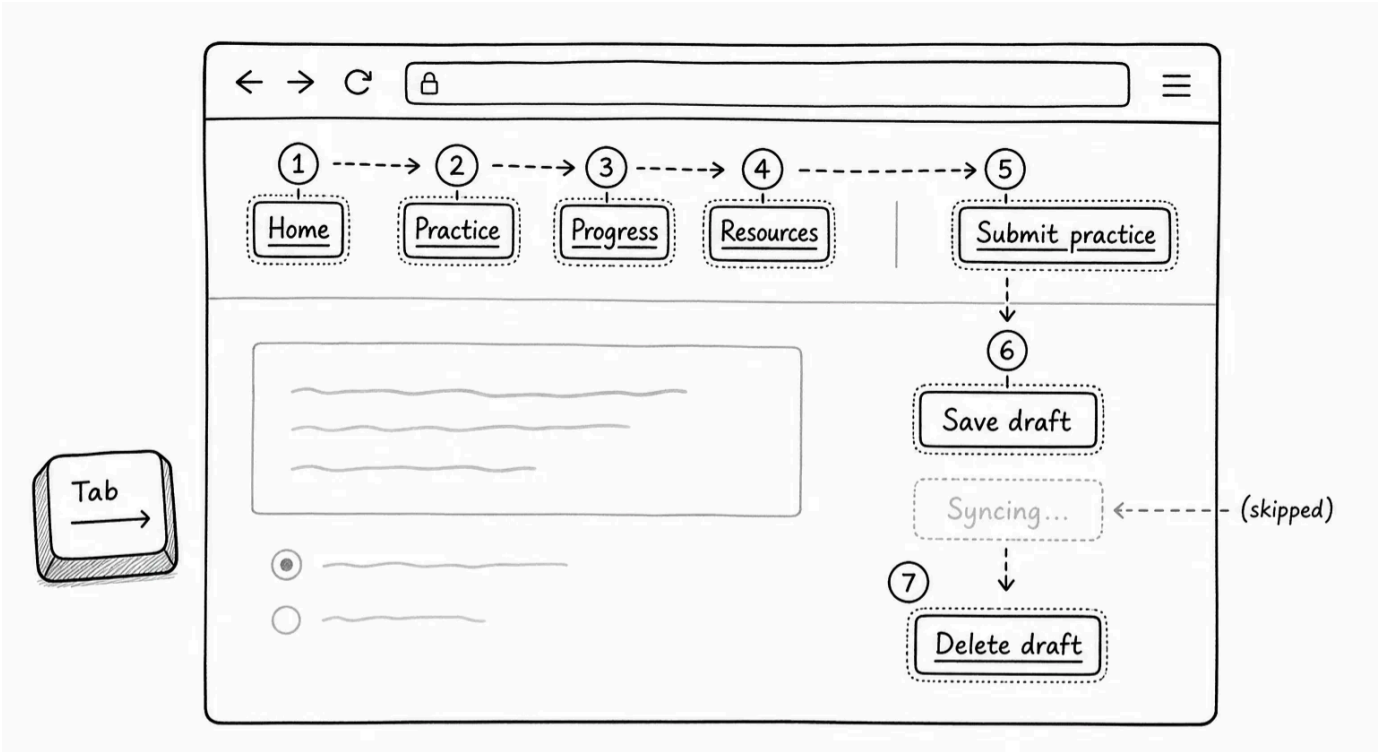
```
a:focus-visible,
button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 3px;
}
```

Reload the page and press `Tab` again. The focused element should be easy to find.

You may also see `:focus` in older CSS. `:focus` matches whenever the element has focus, including after some mouse clicks. `:focus-visible` matches when the browser thinks a visible focus ring is useful, usually during

keyboard navigation. For modern code, `:focus-visible` is usually the better default.

`outline` draws a focus ring outside the element without changing the element's size. `outline-offset` moves the ring away from the edge so it is easier to see.



⚠ Do not remove focus without replacing it - Avoid writing `outline: none;` unless you provide another clear focus style. Keyboard users need to see where focus is.

Add active states

`:active` matches while the user is pressing the element. Add this small pressed state:

```
CSS

.primary-link:active,
.secondary-button:active,
.danger-button:active {
  transform: translateY(1px);
}
```

Click and hold one of the controls, and it should feel slightly pressed.

The `transform` property can move, scale, or rotate an element visually. Here it moves the element down by `1px`. You will learn more about transforms later, but this small use is common for press feedback.

Keep active states subtle because the goal is to show that the click registered, not to make the layout jump around.

Style disabled buttons

The final button has the `disabled` attribute:

html

```
<button class="secondary-button" type="button" disabled>Syncing</button>
```

A disabled button cannot be clicked or focused, so style it with `:disabled` :

CSS

```
button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

button:disabled:hover {
  background-color: white;
}
```

Reload the page and the disabled button should look unavailable.

`opacity: 0.55;` fades the whole button. `cursor: not-allowed;` changes the pointer cue when someone hovers over it. The `button:disabled:hover` rule keeps the disabled button from picking up the normal hover background.

Do not use color alone to communicate disabled state. Lower opacity, cursor change, and text like `Syncing` help make the state clearer.

Links do not have a real `disabled` attribute. If a navigation option should not be available, it is often better to remove the link, explain why it is unavailable, or use a button if the control is truly an action.

Visited links can look different

Browsers can style links the user has already visited with `:visited` :

CSS

```
.panel-text a:visited {
  color: #6d28d9;
}

.panel-text a:hover {
  color: #1d4ed8;
}
```

This can help users remember which pages they have already opened.

The selector above targets normal text links inside `.panel-text` . That keeps the visited color away from button-style links like `.primary-link` , where a purple visited color could clash with the blue background.

There is one cascade detail worth knowing: a link can be visited and hovered at the same time. When state selectors have the same specificity, source order decides. That is why `.panel-text a:hover` comes after `.panel-text`

a:visited in the example. The hover color can still win while the user is interacting with a visited text link.

A common order for link states is **:link**, **:visited**, **:hover**, **:active**, often remembered as LVHA. The **:link** state means an unvisited link. You do not need to write every one of those states today, but it helps to recognize the order when reading CSS.

Browsers limit what you can style with **:visited** for privacy reasons. Color changes are the main safe use. You do not need to use **:visited** for every site, but it is worth recognizing when you see it.

Put the states together

The page now has a small state system:

CSS

```
.panel-text a:visited {
  color: #6d28d9;
}

a:hover {
  color: #1d4ed8;
}

.panel-text a:hover {
  color: #1d4ed8;
}

a:focus-visible,
button:focus-visible {
  outline: 3px solid #93c5fd;
  outline-offset: 3px;
}

.primary-link:hover {
  color: white;
  background-color: #1d4ed8;
}

.secondary-button:hover {
  background-color: #eff6ff;
}

.danger-button:hover {
  background-color: #fef2f2;
}

.primary-link:active,
.secondary-button:active,
.danger-button:active {
  transform: translateY(1px);
}

button:disabled {
  opacity: 0.55;
  cursor: not-allowed;
}

button:disabled:hover {
  background-color: white;
}
```

The important habit is not memorizing every state. It is remembering that interactive elements need visible feedback for different ways people interact with them.

Try it

Before moving on, test the page like a user would. Use the pointer, use the keyboard, and deliberately break a few states so you can feel why they matter:

- Hover over the paragraph link, navigation links, primary action link, and buttons.
- Press **Tab** to move through the interactive elements. Confirm that every focused element has a visible ring.
- Temporarily remove the **a:focus-visible, button:focus-visible** rule. Press **Tab** again and notice how much harder the page is to use. Then add it back.
- Change the hover background on **.primary-link:hover** to a much lighter blue. Check whether the white text is still readable.
- Add **disabled** to the delete button in the HTML and reload. Confirm that **button:disabled** styles it too.
- Add a new navigation link and check whether the base link, hover, and focus styles apply without writing a new class.
- Add another link inside **.panel-text** and check whether the scoped visited style belongs to text links, not button-style links.

The goal is to make interaction visible. If a user can click, tap, or tab to something, its states should be clear.

< 0 / 15 >

Knew

0 Items

Learnt

0 Items

Skipped

0 Items

ResetProgress

Test Your Understanding

When should you use a link?

Click to Reveal the Answer

✔ Already Know that

⚡ Didn't Know that

⏩ Skip Question



LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →